Auditing Predictive Updates from Programming Retries: Target Alignment, Future Outcomes, and Retrospective Risk Screening

Anonymous working manuscript

26 September 2026

Working research manuscript

Abstract

Repeated programming submissions can improve a student model's forecasts without measuring a learning effect. We audit predictions after the first and last submissions of an episode against the identical first observed response to the next problem. The design extends prior future-target evaluations by examining forecast-origin alignment, training-task controls, and retrospective failure screening on two public programming platforms. On a learner-disjoint ACcoding cohort of 1,500 learners and 69,825 pairs, first-attempt logistic training reduces log loss by 0.001566, with a learner-cluster 95% interval of [0.000756, 0.002300] for the reduction. A paired-target control and two three-seed GRU ensembles also improve proper losses, although mean probability revisions differ in sign. CodeBench update effects are uncertain under first-attempt and paired-target training; paired-target fitting improves absolute forecasting scores on a previously unused holdout. At a validation-fixed screening threshold, the GRU first-attempt ensemble improves failure recall by 0.694 percentage points [0.336, 1.057], while actual workload rises from 30.71% to 30.81%. Other operating points show smaller or mixed trade-offs. On a fresh CodeBench semester, intermediate-code features yield an inconclusive log-loss change of -0.001094 [-0.006666, 0.004346] beyond endpoint controls on 274 pairs from 28 learners. These secondary intervals are descriptive and conditional on the fitted pipelines. Results support additional predictive information in the selected ACcoding trajectories and show why probability revisions, task alignment, and decision metrics must be evaluated separately. They do not establish causal learning gains, exercise-selection value, or a state-of-the-art model.

Keywords: student modelling; programming education; repeated attempts; probabilistic forecasting; evaluation validity; learning analytics.

1 Introduction

An accepted programming submission is an operational outcome of a particular interaction with an automated judge. A sequence of submissions also contains information about persistence, local debugging, problem familiarity, and knowledge. Adaptive systems often compress this information into a student state and use the state to predict performance or select an exercise. The validity of that update matters before a downstream objective can be interpreted educationally. Previous programming knowledge-tracing research has already distinguished general knowledge from proximity to solving the current problem. Code-DKT also illustrates forecasts for a fixed future problem as previous interactions arrive [3]. Consequently, neither distinguishing retries from first attempts nor inspecting a future-problem probability constitutes a new contribution on its own. Our question concerns the population-level evaluation of those updates: does additional evidence improve a proper score on the *same* subsequent outcome, how much does the conclusion depend on the forecasting task used during training, and what observable operational trade-off follows? Three quantities require separate interpretations. Success on a repeated problem measures a local observed outcome. A change in a

model probability measures a predictive revision. An effect of assigning an exercise on later proficiency is an intervention contrast. The first two can be investigated with suitably defined observational targets. The third requires additional design or identification assumptions. We therefore evaluate predictive information and retrospective screening, rather than claim an effect on learning from a change in probability. The study addresses three initial research questions. RQ1 asks whether additional submissions improve predictions of an identical subsequent first-attempt outcome. RQ2 asks how this finding changes when training is aligned with the forecast origin and outcome, and when a sequential rather than aggregate-history model is used. RQ3 asks whether the updates alter failure detection and review workload when screening thresholds are selected on validation data and frozen before evaluation. A subsequently specified extension, RQ4, asks whether strictly intermediate source code adds predictive value beyond endpoint-code and feedback/time/count representations at the same late checkpoint, using a fresh semester holdout. The contribution is an empirical audit of forecasting-task alignment, repeated-submission evidence, and an observable screening trade-off on two programming platforms. A recent BKT artifact already describes a controlled same-future-target evidence-integration study [14]; we do not claim to introduce that general design. Standard conditional-expectation and proper-scoring identities provide interpretation, not a new theorem. Logistic regression, recurrent modelling, and validation-based shrinkage serve as controls, not novel architectures.

2 Related work

2.1 Repeated attempts and programming student models

Code-DKT [3] is the closest conceptual precedent. Its first-versus-all-attempt comparison explicitly examines general knowledge versus proximity to solving the current problem. Its Case C also follows the predicted probability of the same future problem as earlier interactions arrive. Our distinction is a systematic paired evaluation of within-problem retry blocks, proper scores on identical subsequent outcomes, coverage accounting, and controls for forecast-origin alignment. We do not claim priority for the fixed-future-problem perspective. Learning Code-Edit Embeddings [4] models program modifications and debugging behavior, including personalized next-step code suggestions. It establishes that changes in code can carry process information. Our primary cross-platform comparison uses observable histories and operational outcomes. CodeBench additionally permits prior-code structural features, but their presence does not establish an expert-labelled misconception or a particular repair mechanism. Code-aware cross-problem prediction has further precedents in HELP-DKT [21] and open-ended programming KT [22], with different target definitions. The srcML-DKT study [20] highlights representation of submissions with syntax errors and evaluates first attempts. The revision-aware success study [19] compares current, paired, and multi-snapshot representations for outcomes on the same problem. These studies motivate a narrower distinction between endpoint code, strictly intermediate snapshots, and feedback/count/time summaries when examining a subsequent different problem. A benefit from a code-difference feature is not necessarily evidence of new information: a difference computed solely from two endpoint programs is a deterministic function of those endpoints. Finite models can nevertheless benefit from that representation. Deep Knowledge Tracing [5] motivates recurrent state models. Our GRU is an adapted question-conditioned baseline with learned event embeddings and a query decoder, rather than an exact reproduction of the original RNN/LSTM implementation. The recent CodeInsight dataset study [6] evaluates iterative programming models with learner and future-week separation and already includes probabilistic losses. Neither learner holdout nor the use of log loss or Brier score is a novelty claim here. CodeInsight access is gated; it is not silently treated as an acquired benchmark. Reliability concerns in programming KT also have direct prior treatment. Kim and Kim [7] examine experimental protocols and attention-related implementation issues. Our invariance checks address the concrete lineage of features and targets in the present pipeline; they do not amount to an independent

reproduction of every reported failure in that work. The paper is cited in its verified arXiv version, whose metadata reports ITS 2026 acceptance. Attempt aggregation is also established. GRATE [18] adaptively aggregates attempt information in complex problem solving, including programming. Pu and Becker [17] examine repeated-attempt removal and episode-based processing when interpreting attention models. Thus compressing retries or changing update frequency would not by itself establish novelty. CE-KT [15] studies correctness-conditioned behavioral evidence, future-outcome validity, proper scores, and permutation/random-proxy controls; it also reports limitations of the additional evidence. Zhang et al. [16] investigate self-regulated-learning indicators and reliability of skill-mastery estimates. For the latter, we verified proceedings metadata and the institutional abstract but could not inspect the full text, so we make no claim about its precise estimator. A particularly close artifact was published on 22 September 2026 [14]. Its public description reports 8,995 scaffold--next-same-skill pairs, fixed BKT parameters and pre-episode states, matched subsequent targets, learner-cluster uncertainty, and a difference between discrimination and probabilistic-score findings. We inspected the dataset/notebook metadata, not the notebook itself; peer-reviewed publication was not verified. Nevertheless, this public description rules out claiming first use of a matched within-model future-target audit. Our narrower extension investigates programming first-versus-last forecast origins, task-aligned fitting, and a fixed-threshold application; the degree of novelty must be judged against these direct precedents.

2.2 Prediction, instructional decisions, and evaluation

The distinction between predictive performance and tutoring utility predates recent deep models. LEOPARD [8] makes tutoring usefulness an explicit evaluation concern, while the Robust Evaluation Matrix [9] tests instructional policies across student models. These studies preclude a broad claim that linking prediction to instructional decisions or checking simulator dependence is itself new. ExRec [10] combines semantically grounded knowledge tracing and exercise recommendation. Its published reward is a signed change in the model's estimate; the source implementation also preserves signed changes. We do not substitute a positive-clipped toy reward for that formula or claim to have reproduced ExRec's policy results. UniER [11] provides an item- and path-level recommendation benchmark. Our study addresses a narrower measurement task before an exercise-selection policy can be assessed: whether a specific student-model update provides useful information on a common future target.

2.3 Probabilistic forecasting foundations

Proper scoring rules evaluate the correspondence between probabilistic forecasts and outcomes [1]. Coherent conditional forecasts and calibration are established statistical concepts [2]. We apply these foundations to retrospective programming episodes and make the forecast origin explicit. Standard information identities serve to constrain interpretation; they are not presented as new results in probability theory. The literature review is a targeted nearest-work audit through 26 September 2026, not an exhaustive systematic review or a proof that no related protocol exists.

3 Problem, forecast origins, and estimands

3.1 Retrospectively defined paired outcomes

For a learner, an episode is a maximal contiguous block of submissions to problem i. We retain episodes with at least two submissions. Let H_0 be the observed history immediately after its first submission and H_1 the history immediately after its final submission. Problem $J=j$ is the problem in the immediately following episode. We require $j \neq i$, a first observed response by this learner to j, and an item represented in the training data. In CodeBench, earlier execution exposure is additionally relevant. Dataset-specific exclusions and coverage must be reported rather than hidden by the

first-attempt label. The binary target Y_j is the same observed outcome for both forecasts. With a frozen predictor f_θ, we compare $p_0 = f_\theta(H_0, j)$ with $p_1 = f_\theta(H_1, j)$. Target code, target result, target grade, and target timestamp are excluded from both feature vectors. The target's identity is conditioned on retrospectively; the audit does not predict which exercise will be selected. Episode endpoints and eligibility are also defined from the released trace, so the design is not a randomized comparison of receiving more attempts. Let $S = 1$ denote eligibility and outcome observability. For loss ℓ, the pair-weighted estimand is

$$\Delta_\ell = \mathbb{E}[\ell(Y_J, p_1) - \ell(Y_J, p_0) \mid S = 1]. \tag{1}$$

Negative values indicate better prediction after the additional observations. The evaluated proper losses are binary log loss,

$$\ell_{\log}(y, p) = -y \log p - (1 - y) \log(1 - p), \tag{2}$$

and the Brier loss $\ell_B(y, p) = (y - p)^2$. We also report mean predictive revision $\mathbb{E}[p_1 - p_0 \mid S = 1]$ as a descriptive quantity. These measures have different scales and meanings; their magnitudes should not be equated.

3.2 Information gain is not an intervention effect

The following standard identity clarifies the target of the audit [2, 1]. Work under the distribution conditional on the same target identity and eligibility event, with nested information sets $\mathcal{F}_0 \subseteq \mathcal{F}_1$ and fixed future outcome Y. If $q_t = \mathbb{E}[Y \mid \mathcal{F}_t]$, then the tower property gives

$$\mathbb{E}[q_1 - q_0 \mid \mathcal{F}_0] = 0. \tag{3}$$

This statement does not require a learner's knowledge to remain constant: both forecasts concern the same future random variable. It does not apply to a comparison of different time-indexed proficiency variables as though they were one target. For the same oracle forecasts,

$$\mathbb{E}[(Y - q_0)^2 - (Y - q_1)^2] = \mathbb{E}[(q_1 - q_0)^2] \geq 0. \tag{4}$$

To see this, expand $Y - q_0 = (Y - q_1) + (q_1 - q_0)$; the expected cross-term is zero by conditional expectation. The analogous expected log-loss reduction is conditional information about Y. These are standard forecasting consequences. Finite fitted models, lossy representations, task mismatch, and distribution shifts need not satisfy oracle properties. Observed mean revision alone therefore does not diagnose a faulty student model. In particular, rolling response summaries are not nested information sets even though the full underlying histories are nested. No potential-outcome contrast between alternative assigned exercises is identified by this design. Conditioning on learners who continue, their observed next exercise, and the availability of its outcome leaves selection and censoring unresolved. Better forecasts demonstrate useful information for the stated conditional prediction problem, rather than knowledge acquisition caused by the preceding exercise.

3.3 Forecast-origin alignment

A model trained to predict the next submitted response is ordinarily queried immediately before that response. Applying it at H_0, before several additional submissions on another problem, changes the prediction task. Restricting its training labels to first attempts does not fully align this earlier forecast origin. We therefore include a paired-target control: training learners contribute both H_0 and H_1 vectors with the same future label, together with an observed phase indicator. Each phase receives half of a pair's weight. The scaler and logistic model are fit only on training learners and frozen for evaluation. The phase coefficient may mechanically improve stage-specific calibration on the training population. It is not evidence of a new learning mechanism or a recovered latent knowledge state. The control tests the practical consequence of matching the training task to the retrospective audit.

Held-out scores, rather than a desired direction of probability revision, determine its empirical usefulness.

4 Data and experimental design

4.1 Sources and provenance

ACcoding was downloaded from the public Zenodo record labelled version 2.0.0 [12]. The verified archive contains 2,181,602 submission records from 9,866 learner identifiers and 4,301 problem identifiers; 2,137,950 records are labelled C. Observed timestamps span January 2024 through December 2025, which does not match the record description's May 2025 cutoff. We retain both the published metadata and the measured file statistics. The release contains verdicts, timestamps, identifiers, and metadata, but not full submission source code or per-test outcomes. The CodeBench 2016-1 archive [13] contains 471 unique learner identifiers across 535 learner--class enrollments, nine classes, and 235 exercise identifiers. It contains 29,991 Python source files and the same number of execution logs. The audited execution histories include 167,791 TEST and 153,931 SUBMITION events, preserving the source's spelling. Of the 471 identifiers, 466 have submissions, one has only TEST events, and four have no dated events. The submission learners split into 329 training, 74 initially reserved validation, and 63 test users. Actual timestamps extend from May 2016 into October 2017; the semester label is not a temporal cutoff. Splits group learner identifiers across classes. All 105,052 submissions with numeric grades have no ERROR section, whereas all 48,879 missing-grade submissions contain an ERROR section in the audited release. The primary target distinguishes grade 100 from lower numeric grades, excluding missing target grades after determining first observed submission status. A separately reported operational definition counts a missing grade accompanied by ERROR as unsuccessful. Missing-grade events remain explicit missing states in the history under both definitions. We do not equate an execution error with a known numeric grade of zero. Code-feature controls use only earlier source snapshots and syntactic counts, not expert repair labels or target code. The two platforms provide different operational outcomes, interaction interfaces, languages, and periods. Their results are presented separately. Replication of a qualitative pattern across them would not identify the cause of a domain difference. Public accessibility also does not establish unrestricted redistribution rights. Source URLs, hashes, parsers, and measured schemas accompany the local analysis package.

4.2 Exploration, independent cohort, and controls

The initial ACcoding pilot selected 1,500 learners with at least 20 distinct problems in the released trace, retaining 424,311 submissions. Stable learner hashes assigned 1,054 learners to training, 212 to validation, and 234 to the exploratory test group. Eligibility uses complete trajectories and defines a selected active population. The exploratory test results informed the study design and are not presented as an untouched confirmatory test. An additional learner-disjoint cohort of 1,500 learners contains 420,076 submissions. All original pilot learners were excluded from its candidate pool, and identifiers were locked before extracting the selected histories. Training and validation remain the original groups. Amendments were recorded before evaluation of the new cohort; these internal records are not a claim of external preregistration. The primary replication contrast is the first-attempt-trained model's full update against its frozen early prediction under log loss. Additional model, subgroup, and application analyses are secondary. Aggregate-history models use training-derived item difficulty, history count, smoothed accuracy, last response, recent response accuracy, and the number of distinct prior problems. Item difficulty is estimated from training learners' first observed responses, with smoothing and learner-fold cross-fitting for training examples. No elapsed time to the future target is used in the paired audit. Logistic models contrast all-submission training, first-attempt training, and paired-target training. The sequential control is an explicitly adapted question-conditioned GRU; it is

not described as an exact reproduction of a published state-of-the-art system. Two secondary controls were specified after the main study design. In CodeBench, the originally unused 74-learner validation partition becomes a new holdout for a fixed paired-task comparison, with no tuning on that partition. The original 63-user test results remain separately reported. A calendar-separated ACcoding check fixes the 80th percentile of original training-event timestamps as a cutoff, fits first-attempt parameters and item statistics only at or before that cutoff, and retains new-cohort pairs whose early and late checkpoints are at or after that cutoff, with the target strictly after the late checkpoint. This check follows inspection of the main cohort results and is exploratory, not another independent confirmation. Chronology concerns recorded submission times; judge feedback-completion timestamps are unavailable.

4.3 Intermediate-code extension on a fresh semester

To distinguish coarse retry evidence from recorded code content, a later extension uses CodeBench's official 2016-2 archive. It contains 172 archive learner identifiers, 38,881 submissions, and 87,299 TEST events across five classes; recorded events span October 2016 to February 2017. Every submission has exactly one nonempty CODE section. All 10,888 missing grades coincide with an ERROR section; the 27,993 numeric grades do not. These counts are schema observations, not model-performance results. We exclude all 38 identifiers overlapping the original archive from every fresh-cohort partition. A fixed learner hash assigns the remaining identifiers to training, validation, and test using nominal 60%/20%/20% ranges. Of 132 new identifiers with submissions, the realized counts are 79, 23, and 30. Original 2016-1 training users supplement the fresh training partition; neither old validation nor old test users are used. Item identifiers are namespaced by semester because cross-semester equivalence was not established. This pooled design is retrospective, not calendar-forward transfer, and disjoint identifiers do not prove different real people. Eligible source episodes contain at least three timestamped submissions from one execution-log path. Source and target submission timestamps must be unique within the learner's recorded second. The immediately following target is the first observed submission to a different, training-supported problem, with no earlier TEST/SUBMITION exposure at or before H_1. First-submission status is assigned before requiring a numeric target grade; a later graded retry never replaces a missing first grade. The primary label is grade 100 versus a lower numeric grade. Source missing grades and syntax-invalid programs remain represented. Target TEST activity after H_1 is permitted and reported. No student program is executed. All arms predict the identical target at H_1, so this extension compares nested input representations rather than early-versus-late forecasts. M0 has fixed history, source/target identity and difficulty, semester, and source feedback/count/time summaries. M1 adds first/final code representations and endpoint-difference features. M2 adds strictly intermediate code representations and normalized path summaries. A secondary endpoint-redundant control adds the mean endpoint token vector to M1. This matches M2's extra token-vector dimension but not its additional numerical features, and is therefore only a partial regularization/capacity control. The sole primary extension contrast is validation-calibrated test NLL for M2 minus M1. A prespecified magnitude of 0.005 nats per target is an analyst-chosen predictive threshold, not educational utility. The vocabulary, scalers, item statistics, and regressions use designated training learners only. Each arm uses the same fixed logistic fitting rule and scalar validation calibration. No feature, hyperparameter, outcome, or subgroup search follows test evaluation. The locked protocol and feature registry accompany the implementation. These representations are lossy. The experiment can assess incremental predictive value relative to the specified pipeline; it does not condition on all raw endpoint/feedback information or estimate conditional mutual information of the underlying code. The CODE snapshots are recorded alongside submission events. Their extraction and hash alignment do not independently establish exactly which code the grader executed or when feedback became available. Text equality is also not a ground-truth no-learning control: unchanged code can accompany different recorded grading outcomes for reasons not identified by these logs.

4.4 Update control, uncertainty, and checks

A simple control interpolates between forecasts:

$$p_\alpha = p_0 + \alpha(p_1 - p_0), \qquad \alpha \in \{0, 0.25, 0.5, 0.75, 1\}. \tag{5}$$

The validation set selects α by log loss, with a fixed tie rule. The selected value is frozen for the independent cohort. This is a transparent baseline for the value of the update; interpolation itself is not proposed as a new algorithm. Uncertainty intervals resample learners while retaining paired observations. Pair-weighted point estimates give more weight to learners with more eligible pairs; learner-cluster bootstrapping addresses dependence, not this weighting choice or selection bias. Intervals condition on fitted models and a fixed split. Recurrent-model seed variation is reported separately. Secondary intervals are descriptive and do not imply family-wise error control. Programmatic checks test prefix construction, identity of the target and its difficulty, invariance to changes in the target or later outcomes, and agreement between the late forecast and the model's ordinary forecast at that target. Tied timestamps are documented and handled according to each parser's protocol. Student code is parsed as data and is not executed.

5 Retrospective application: failure screening

The application flags an observed future first attempt as at risk using $r = 1 - p$. For each model and checkpoint, validation data select the highest threshold attaining at least 80% failure recall and, separately, a threshold at the validation top-30% risk quantile. Ties are included. Test thresholds remain unchanged; the 30% rule therefore need not produce exactly 30% workload on the new cohort. We report the flagged fraction, failure recall, missed-failure fraction, and positive predictive value. The same outcomes and eligible pairs are used to compare early and late forecasts. Their thresholds are calibrated independently to the same validation criterion, so achieved test recall and workload must be interpreted jointly. These quantities describe an observable detection--workload trade-off under the selected retrospective task. They do not measure whether an educator's review would prevent failure, whether the learner would accept an alternative exercise, or whether an intervention improves learning. The future exercise identity is known in this evaluation. Application results therefore support an audit or screening use case, not causal exercise recommendation.

6 Results

6.1 Independent-cohort coverage and predictive information

The independent ACcoding cohort contains 83,303 multi-submission episodes. We exclude 735 without an observed next episode, 12,492 whose next problem was previously observed, and 251 whose target item was absent from training. The remaining 69,825 pairs involve all 1,500 learners, cover 83.82% of multi-submission episodes, and represent 38.82% of first observed problem responses in this cohort. Their acceptance rate is 46.77%. We retain 16,739 source episodes ending without acceptance; selecting only successful episodes is not part of the primary analysis.

Table 1. Independent ACcoding cohort: the same 69,825 future outcomes under early and late forecasts. Lower NLL is better. These are task-specific controls, not a state-of-the-art leaderboard.

Training task	NLL at H_0	NLL at H_1	Mean $p_1 - p_0$
All submissions	0.58915	0.58705	0.049427
First attempts	0.58112	0.57955	0.024789
Paired future target	0.58045	0.57903	-0.000153

For the prespecified first-attempt model, the late-minus-early NLL difference is -0.001566, with a 1,000-replicate learner-cluster 95% interval of $[-0.002300, -0.000756]$. The Brier difference is -0.000665 $[-0.000980, -0.000333]$. This replicates a small predictive-information benefit on learners not used in the exploratory pilot. It is not an estimate of knowledge gained by those learners. The all-submission predictor also improves, with an NLL difference of -0.002099 $[-0.003262, -0.000829]$. The task-aligned paired-target predictor improves by -0.001416 $[-0.001842, -0.000971]$, with a Brier difference of -0.000593 $[-0.000779, -0.000405]$. Thus a benefit remains after controlling for the earlier forecast origin. The models are fit to different training tasks, so comparisons of their absolute risk also reflect task alignment and training population rather than architecture alone.

6.2 Predictive revision and validation-selected shrinkage

Mean probability revision differs substantially across the controls (Table 1). For the paired-target model, its interval is $[-0.000372, 0.000063]$, despite the reduction in proper losses. The near-zero mean is partly anticipated by the model's phase-specific calibration and is not evidence that a latent learning estimate has been corrected. The result shows why the size or sign of an average probability change should not substitute for a direct evaluation on future outcomes. Validation selected update weights of 0.50, 0.75, and 1.00 for all-submission, first-attempt, and paired-target training, respectively. The first-attempt model's selected update achieves NLL 0.57913 and Brier 0.19795. Its NLL improvement relative to the full update is -0.000418 $[-0.000614, -0.000236]$. Ten-bin ECE changes from 0.01170 to 0.00767. This modest improvement is an out-of-sample result for a simple baseline, not a new optimization method. Shrinkage reduces the all-submission predictor's NLL by 0.001425 relative to its full update, while validation selects no shrinkage for the paired-target control. Calibration metrics do not move uniformly: all-submission ECE increases from 0.03670 to 0.03977 despite improved NLL. ECE is bin-dependent and does not replace proper scores. The complete calibration bins and all predefined subgroup results are retained in the analysis package.

6.3 CodeBench: training task and outcome definition matter

The original CodeBench test split contributes 1,125 numeric-grade pairs and 1,665 operational-outcome pairs, each from 61 learners. These represent 45.27% and 67.00% of its 2,485 multi-submission episodes, respectively. The numeric analysis excludes 540 structurally eligible pairs whose target has no numeric grade. Earlier TEST exposure excludes a target before the late checkpoint, but 442 numeric pairs and 593 operational pairs have a TEST event between that checkpoint and the first submitted target response. The target is therefore a first *submission*, not necessarily an unaided first execution.

Table 2. CodeBench history predictors: late-minus-early NLL, with 2,000-replicate learner-cluster intervals. "All" and "first" refer to training labels. Each outcome definition is a separate conditional prediction task.

Outcome / training	Pairs	Delta NLL	95% interval
Numeric / all	1,125	-0.044019	[-0.073846, -0.012115]
Numeric / first	1,125	-0.005277	[-0.016748, 0.006288]
Operational / all	1,665	-0.025970	[-0.052860, 0.001490]
Operational / first	1,665	-0.001912	[-0.010907, 0.006398]

The larger improvement under all-submission numeric-grade training is not reproduced with comparable certainty by the first-attempt models. Their intervals include both improvement and no benefit (Table 2); this is uncertainty, not proof of a null effect. Adding prior-code structural features gives first-attempt NLL changes of -0.008520 for numeric targets and -0.003602 for operational

targets, with intervals containing zero in both cases. The difficulty-only control is exactly invariant between checkpoints, as expected. The mean probability change is 0.143429 for all-submission numeric training but 0.004619 for first-attempt numeric training. Under the operational definition, the corresponding first-attempt change is -0.001899. Its direction also depends on pair versus equal-learner weighting near zero. These observations concern model outputs; none is a measured increase or decrease in student proficiency. Timestamp-batch state construction retains 25,621 submission rows in tied-time groups while excluding ambiguous probe boundaries. A sensitivity analysis removes the one learner with a detected within-file clock reversal, removing 22 numeric and 27 operational pairs. The qualitative NLL interval conclusions remain unchanged. Equal-learner weighting also leaves the first-attempt NLL contrasts inconclusive. The full subgroup, weighting, code-feature, and clock-sensitivity outputs are retained rather than selected according to significance.

6.4 CodeBench task-aligned training on a new holdout

The previously unused 74-learner partition contributes 1,342 numeric-target pairs and 2,033 operational-target pairs from 72 learners. The ordinary first-attempt comparator is fit to 14,895 numeric or 19,474 operational training examples. Paired-target fitting uses 5,405 or 7,801 training pairs, respectively, each contributing two half-weighted snapshots. The models share 11 historical features; only the paired model adds the checkpoint indicator. These task-specific training populations differ, so the comparison is not a pure intervention on the phase coefficient.

Table 3. Secondary CodeBench control on the new holdout. Intervals use 1,000 learner-cluster resamples. Negative changes indicate improved prediction after retries.

Outcome / training	NLL H_0	NLL H_1	Change [95% interval]
Numeric / first	0.62914	0.63204	0.002902 [-0.007076, 0.012731]
Numeric / paired	0.62067	0.62004	-0.000630 [-0.005191, 0.004365]
Operational / first	0.61705	0.62086	0.003813 [-0.004577, 0.012206]
Operational / paired	0.60326	0.60387	0.000609 [-0.002897, 0.004335]

Paired-target fitting improves absolute NLL at both checkpoints for both outcome definitions. At the late checkpoint, its contrast against ordinary first-attempt fitting is -0.011996 $[-0.022986, -0.002108]$ for numeric targets and -0.016994 $[-0.025805, -0.008647]$ for operational targets. However, every within-model update interval contains zero (Table 3). Direct differences between the models' update effects are also uncertain: -0.003532 $[-0.010008, 0.003342]$ and -0.003205 $[-0.008349, 0.002288]$, respectively. Better task-aligned forecasts therefore do not establish that the additional retry observations themselves improve this holdout task. Equal-learner weighting changes some point-estimate signs while leaving all update intervals inclusive of zero.

6.5 Retrospective failure screening

At validation thresholds selected for 80% failure recall, the first-attempt predictor's late forecast changes confirmation workload by -0.016 percentage points, with interval $[-0.167, 0.137]$, and failure recall by 0.046 points $[-0.143, 0.237]$. The observed proper-score improvement therefore does not translate into an established operational improvement at this operating point.

Table 4. Frozen validation-recall thresholds on the independent ACcoding cohort. Values are percentages. Workload is the flagged fraction; recall is the fraction of observed failures flagged. Differences must be assessed jointly.

Model	Workload H_0	Workload H_1	Recall H_0	Recall H_1
All submissions	64.91	64.47	81.51	81.15
First attempts	64.56	64.54	81.26	81.31
Paired target	64.60	64.74	81.30	81.44
GRU ALL ensemble	65.73	65.10	80.95	81.12
GRU FIRST ensemble	66.47	65.85	81.63	81.43

For the all-submission predictor, workload decreases by 0.444 points $[0.211, 0.656]$, but failure recall also decreases by 0.358 points $[0.090, 0.631]$. Calling only the workload decrease a benefit would omit the loss in detection. The paired-target model uses slightly more workload; its recall-change interval includes zero. These intervals are secondary and descriptive, conditional on the fitted models and thresholds. At the frozen validation 30% workload thresholds, confirmation workloads are approximately 30.45--30.85% and failure recalls 45.31--45.77%. Every logistic model's late-minus-early recall interval includes zero under that rule. This application evaluates observable trade-offs, rather than infer decision usefulness from NLL alone. It does not test an intervention or an exercise-selection policy. The recurrent ensembles provide a modest positive screening result under that second rule. GRU FIRST failure recall rises from 44.58% to 45.27%, a change of 0.694 percentage points $[0.336, 1.057]$, while workload changes from 30.71% to 30.81%, or 0.102 points $[-0.141, 0.337]$. Its positive predictive value rises from 77.27% to 78.21%. GRU ALL recall changes by 0.350 points $[0.005, 0.704]$, with workload changing by -0.083 points $[-0.313, 0.158]$. These are secondary nominal intervals, not multiplicity-corrected evidence. The rule transfers a validation threshold; it does not hold confirmation workload exactly equal. An interval spanning zero for workload cannot establish workload equivalence. Under the validation-80%-recall rule, the ensembles reduce workload by approximately 0.62 points each, but their recall-change intervals contain zero: ALL 0.175 $[-0.137, 0.540]$ and FIRST -0.202 $[-0.468, 0.059]$. No noninferiority margin was specified. We therefore report the joint changes without claiming that detection is preserved. Across all five predictors, validation and confirmation pair identities and labels match exactly; differences do not arise from distinct evaluation populations.

6.6 Calendar-separated check

The cutoff is 27 October 2025 at 08:45:57 in the source's unspecified timezone. Past-only fitting uses 236,631 original-training events and 103,285 first-attempt labels. Evaluation retains 2,291 pairs from 516 learners in the new cohort. The earliest retained early checkpoint is later than the latest parameter-source event. NLL decreases from 0.650422 to 0.646527, a change of -0.003895 $[-0.007585, -0.000100]$; Brier loss decreases by 0.001845 with a change interval $[-0.003636, -0.000017]$. This secondary check is consistent in direction with the main first-attempt result, with marginal interval separation from zero. Coverage sharply limits the interpretation. Past-only item support excludes 11,130 candidate episodes, versus 251 under full-period training support. Of 58,946 otherwise eligible past-supported pairs, 56,655 have early checkpoints before the cutoff. The final subset is only 3.89% of those pairs. This is evidence for a selected later, previously supported item population, not broad prospective deployment performance. A nullable-Boolean episode-boundary error was repaired before computing metrics; the original cutoff and frozen parameter/reference hashes remained unchanged.

6.7 Sequential model check

The validation-converged GRU evaluates the same 69,825 pairs from 1,500 new learners. Both loss modes consume the complete past interaction sequence. Table 5 reports the prespecified mean-probability ensemble of seeds 11, 29, and 47; it is not the mean of seed-specific losses or a selected best seed.

Table 5. Adapted GRU confirmation. Intervals use 1,000 learner-cluster resamples conditional on the fitted ensembles.

Training labels	NLL H_0	NLL H_1	Change [95% interval]
All submissions	0.594697	0.588198	-0.006499 [-0.007939, -0.005156]
First attempts	0.596264	0.592048	-0.004215 [-0.005637, -0.002781]

The corresponding Brier changes are -0.002846 $[-0.003486, -0.002259]$ and -0.002013 $[-0.002607, -0.001429]$. All six individual-seed NLL-change intervals are below zero. Thus the ACcoding predictive-information finding is not restricted to aggregate-history logistic models in these experiments. Mean probability revision is 0.016873 for ALL training but -0.002783 for FIRST training, although both ensembles improve proper losses. FIRST seed 47 has a small positive mean revision, whereas seeds 11 and 29 are negative; optimization variability remains relevant. A negative average revision is not evidence of forgetting, just as a positive revision is not a measured learning gain. Moreover, a larger within-model loss reduction does not establish a better final predictor: the GRU late NLLs here remain higher than the aggregate-history first-attempt and paired-target logistic values. No matched SOTA comparison or general architectural ranking follows from these results.

6.8 An exact interpretation check

An accompanying nine-case enumeration provides a sanity check for the forecasting identities. Let initial knowledge be Bernoulli with probability 0.4, let an unmastered skill be learned with probability 0.15 before an observed response, and let response success probabilities be 0.9 and 0.2 for the mastered and unmastered states. The same future response has probability 0.543 before the intermediate observation. Knowledge increases by 0.09 in expectation, yet the expected update of a correctly aligned forecast for that fixed future response is exactly zero. Its Brier improvement from observing the intermediate response is 0.060424. Using the immediate-response baseline 0.48 instead produces a mean difference of 0.063 because the target timing differs. These values are exact-enumeration illustrations under specified assumptions, not fitted parameters or student results. Across nine combinations, fraction-arithmetic checks verify the mean-revision and Brier identities; log-loss improvement agrees with mutual information to numerical precision. They illustrate the interpretation problem without claiming a new theorem or invalidating every reward based on an evolving latent mastery estimate.

6.9 Intermediate code beyond endpoint and behavioral representations

The extension fits on 3,906 pairs from 380 contributing training identifiers: 3,177 pairs from 309 old-training contributors and 729 pairs from 71 fresh-training contributors. Validation calibration uses 208 pairs from 20 learners. The fresh test retains 274 pairs from 28 learners. All arms use identical rows and the same future target. The common vocabulary uses 25,962 training source snapshots; fitting converges for all four fixed logistic arms.

Table 6. Fresh CodeBench semester: nested-input forecasts at H_1. Calibrated NLL supplies the prespecified primary comparison. M0 contains nuisance summaries; M1 adds endpoints; M2 adds intermediate code. The redundant control adds endpoint-derived token information only.

Arm	Raw NLL	Calibrated NLL	Calibrated Brier
M0	0.623534	0.626371	0.218678
M1	0.625941	0.627958	0.219048
M2	0.633867	0.626864	0.218568
M1 + redundant token block	0.630323	0.629748	0.219804

The primary calibrated M2-minus-M1 NLL contrast is -0.001094 $[-0.006666, 0.004346]$. Its interval includes both zero and the prespecified improvement magnitude of -0.005. The experiment therefore establishes neither an incremental benefit nor that a benefit of that magnitude is absent. Equal-learner weighting changes the point estimate to 0.000693 $[-0.006256, 0.007408]$, with the same inconclusive interpretation. Secondary findings do not resolve the uncertainty. Calibrated Brier changes by -0.000480 $[-0.002889, 0.002008]$. Relative to the endpoint-redundant arm, M2's calibrated NLL contrast is -0.002884 $[-0.009326, 0.003391]$. Without validation calibration, M2-minus-M1 is 0.007926 $[-0.002876, 0.018107]$. M0 has the lowest point NLL in both raw and calibrated views, but this table does not establish its statistical superiority. Validation calibration does not uniformly improve held-out losses. All specified views are retained, rather than select whichever favors intermediate code. Coverage and measurement limit the question. Of 398 structurally eligible fresh test episodes, 281 have a usable first numeric target grade, and training item support retains 274. Of the retained targets, 211 have TEST activity after H_1 before the first submitted response. Only 28 learners contribute, and their classes overlap training classes. The result is a small within-setting comparison of fixed representations, not an out-of-class or competence-transfer test. No additional feature/model/subgroup branch was fitted after opening this holdout.

7 Discussion and threats to validity

The central interpretation distinguishes additional predictive information, correct alignment of a forecast task, and educational effects. An update can be useful even when its average probability change is close to zero. A large average revision can reflect task calibration or selection and does not establish a large knowledge gain. Conversely, a lack of predictive improvement in one fitted model does not establish that the learner failed to learn. The audit conditions on continued activity and the next observed exercise. First observed responses may follow exposure outside the released trace. Outcome observability can depend on errors or engagement, particularly when a judge does not record a numeric grade. An accepted verdict is not an independently assessed measure of transferable competence. Class structure, problem-selection mechanisms, help use, and AI assistance may be unobserved. Features follow the released record order. ACcoding supplies submission creation times but not verified verdict-release times, so the pipeline cannot establish that every earlier judge result was already available to a real-time system at the queried instant. Prefix invariance checks establish exclusion of later recorded responses and target labels, not asynchronous feedback availability. This limitation concerns both the main and calendar-separated analyses. Learner-disjoint splits assess generalization to new learners within the released setting, not a strictly prospective deployment. In the main analysis, item statistics derived from training learners may include calendar observations later than an evaluated test event. The secondary past-only control reduces this issue on a much smaller selected population; its existence does not retroactively make the main design prospective. Different languages and platform periods do not by themselves constitute a controlled language-transfer study. Small model differences and a limited family of adapted baselines cannot support a general

state-of-the-art claim. This study extends existing evaluation ideas; it does not establish priority for matching future targets, proper scoring, learner-level uncertainty, or separating prediction from instructional utility. The recent BKT artifact and related programming studies substantially narrow any methodological-novelty claim. Neither another architecture nor a larger experiment count would by itself resolve that limitation. The present evidence supports a reproducible empirical audit, with a modest ACcoding finding and uncertain CodeBench update effects, rather than a broadly validated new learning algorithm. The code-content extension tests a more specific question, but its small fresh holdout cannot resolve whether intermediate snapshots improve next-problem forecasts beyond the chosen endpoint and behavioral representations. Token/AST summaries do not exhaust code semantics, and coarse feedback summaries do not remove every alternative explanation. The inconclusive result neither contradicts prior evidence that code can help student modelling nor supports a universal claim that revision paths are useless. A future independent study would need adequate precision and stronger measurement or representation controls, rather than reuse this test for unrestricted search. For recommendation, the next scientific step is to evaluate a specified decision objective on a documented candidate set. A change in a ranking is a stability result until an appropriate utility is established. Observational logs without a justified assignment mechanism do not identify a new policy's educational value. The present screening analysis has a fully observed target but still does not establish intervention efficacy.

8 Conclusion

Additional submissions modestly improve prediction of a fixed subsequent outcome on a learner-disjoint ACcoding cohort, across aggregate-history controls and converged recurrent baselines. CodeBench does not provide comparably clear evidence of an incremental update benefit under the more closely aligned tasks, although task-aligned fitting improves absolute forecasts. A fresh-semester comparison of intermediate code against endpoint and behavioral representations remains inconclusive. Screening changes depend on the predictor and operating point: recurrent ensembles yield small improvements under a validation-workload rule, while other comparisons remain uncertain or trade lower workload for lower detection. These findings extend existing evaluation approaches with an auditable programming study. They support measuring future predictive value and operational trade-offs directly, without interpreting the direction of a probability revision as learning or claiming an untested benefit from exercise recommendation.

Data, code, and research integrity

The working package contains source manifests, processing scripts, internal protocols, aggregate results, and manuscript source. Third-party raw records are referenced through their original providers and are not presented as a newly licensed open dataset. Internal prediction caches, if needed for aggregate analyses, retain pseudonymous grouping keys and are not intended for public release. No new learners were recruited or contacted. Institutional review status and final data-sharing permissions have not been established for this project; no ethics approval or exemption is claimed.

Declaration of generative AI use

OpenAI Codex assisted with literature retrieval, study design, code generation, execution orchestration, methodological checking, and manuscript drafting. This working draft does not assert that final human-author verification has occurred. Before submission, the named human authors must inspect the evidence, validate the analysis and citations, revise the manuscript, and take responsibility for its content. No AI system is listed as an author.

A Implementation and reproducibility details

The analysis environment uses Python 3.13.7, NumPy 2.3.2, pandas 2.3.3, scikit-learn 1.9.0, and PyTorch 2.12.0 on CPU. Dataset archives, extraction manifests, internal protocols, model-selection logs, and aggregate outputs have recorded checksums. The source package provides execution order and distinguishes reference outputs from a fresh rerun. No downloaded student program is executed. These artifacts support inspection and rerunning; they are not an independent external reproduction. ACcoding logistic predictors use seven features: the logit of smoothed first-attempt item accuracy, log prior-response count, smoothed prior accuracy, last response, mean accuracy over the preceding 5 and 20 responses, and log distinct prior-problem count. Training item statistics are cross-fitted by learner; evaluation statistics use original training learners only. Item probabilities shrink toward the training base rate with 20 pseudo-observations. Scaling and L2 logistic regression use training data, $C = 1$, a maximum of 500 iterations, and seed 26092026. No gap to the future target is supplied. All-submission and first-attempt fitting use 295,788 and 127,005 labels; paired-target fitting uses 49,019 pairs represented by 98,038 half-weighted rows. The CodeBench history predictor has 11 features: item difficulty, log history count, known-grade history accuracy, missing-grade fraction, most-recent timestamp-batch known accuracy and missingness, the corresponding preceding-5 and preceding-20 summaries, and log distinct-problem count. Missing outcomes remain explicit in history. Its separate code-feature controls summarize prior code length, lines, parseability, AST nodes, branches, loops, and functions; they are not code-edit embeddings or a direct reproduction of Code-DKT. The recurrent baseline updates $h_t = \mathrm{GRU}(e(q_t, y_t), h_{t-1})$ and decodes a queried item using

$$p_t = \sigma(w_2 \tanh(W_1[h_{t-1}; v(q_t)] + b_1) + b_2). \tag{6}$$

Event and query embeddings have dimension 48, recurrent state and decoder hidden layers dimension 64, and the training vocabulary contains 2,624 items. There are 407,233 parameters. Unknown query and response-event embeddings are fixed to zero; unknown historical events remain represented in sequence length. Both ALL and FIRST modes consume all past events; FIRST only masks which outcomes contribute training loss. This differs from dropping retries from the input. Adam uses learning rate 0.001, betas 0.9 and 0.999, epsilon 10^{-8}, zero weight decay, no dropout, and gradient norm clipping at 5. Batches contain at most 32 length-grouped learners; truncated backpropagation uses 128-event windows. Each nonempty window's selected labels define its mean loss and optimizer step; this weighting is retained for both modes. Hidden states carry across windows and full histories at inference. Seeds 11, 29, and 47 are all reported, with an equally weighted mean-probability ensemble specified in advance. An initial five-epoch cap left validation curves improving. Before opening the new confirmation cohort for these models, a recorded amendment set a 30-epoch maximum, patience four, and improvement tolerance 0.0001. First-attempt validation NLL selects checkpoints; all six checkpoint hashes are frozen before confirmation evaluation. Selected epochs are 13, 11, and 11 for ALL and 8, 11, and 11 for FIRST. The original capped runs are preserved as exploratory artifacts and do not replace these final fits. Prefix/target-label invariance, late-query agreement, and full-versus-chunk state checks are included in the model records. The intermediate-code extension has 46 nuisance numerical features, 23 endpoint numerical features, and 25 intermediate numerical features, with source/target item one-hot categories and a binary semester indicator. One common training-source vocabulary contains at most 2,048 Python-token unigrams; sublinear TF, smooth IDF, and per-snapshot L2 normalization are fixed. Identifier and literal values are retained, comments/spaces omitted, and layout markers preserved; failed tokenization has a lexical fallback and failure marker. Strictly intermediate token vectors are averaged without renormalization. Numerical scalers and categorical levels use training data. Each arm uses liblinear logistic regression with $C = 1$, 2,000 maximum iterations and seed 1729. Scalar validation-logit calibration uses logistic regression with $C = 10^6$, lbfgs, and the same iteration cap. Raw scores, Brier, ECE, endpoint-only improvement, the redundant-token control, and equal-learner weighting are secondary. All contrasts share 1,000

paired learner resamples; source-window TEST summaries omit the entire final boundary second because within-second TEST ordering is not established.

Table 7. Individual GRU seeds on the confirmation pairs. Descriptive intervals condition on each fitted run. Ensemble losses are computed after averaging probabilities and need not equal the mean of these contrasts.

Mode / seed	Mean change in p	NLL change	95% interval
ALL / 11	0.018972	-0.009800	[-0.011832, -0.008021]
ALL / 29	0.014941	-0.009607	[-0.011256, -0.008083]
ALL / 47	0.016707	-0.008424	[-0.010173, -0.006673]
FIRST / 11	-0.002972	-0.006931	[-0.008793, -0.005214]
FIRST / 29	-0.006100	-0.011098	[-0.012959, -0.009141]
FIRST / 47	0.000723	-0.010012	[-0.011950, -0.007973]

References

[1] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359--378, 2007. https://doi.org/10.1198/016214506000001437.

[2] A. P. Dawid. The well-calibrated Bayesian. *Journal of the American Statistical Association*, 77(379):605--610, 1982. https://doi.org/10.1080/01621459.1982.10477856.

[3] Y. Shi, M. Chi, T. Barnes, and T. Price. Code-DKT: A code-based knowledge tracing model for programming tasks. In *Proceedings of the 15th International Conference on Educational Data Mining*, pp. 50--61, 2022. https://doi.org/10.5281/zenodo.6853105.

[4] H. Heickal and A. Lan. Learning code-edit embeddings to model student debugging behavior. In *Artificial Intelligence in Education*, pp. 91--98. Springer, 2025. https://doi.org/10.1007/978-3-031-98465-5_12.

[5] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, vol. 28, 2015. https://papers.nips.cc/paper/2015/hash/bac9162b47c56fc8a4d2a519803d51b3-Abstract.html.

[6] F. Patel, S. T. Truong, D. Q. Nguyen, K. Fukuhara, B. W. Domingue, S. Koyejo, and N. Haber. A dataset for modeling iterative problem-solving. *arXiv:2609.00940v1*, 2026. Author metadata reports EMNLP 2026 Findings. https://arxiv.org/abs/2609.00940v1.

[7] J. Kim and H. Kim. Ensuring reliability in programming knowledge tracing: A re-evaluation of attention-augmented models and experimental protocols. *arXiv:2605.04727v1*, 2026. Author metadata reports ITS 2026 acceptance. https://arxiv.org/abs/2605.04727v1.

[8] J. P. González-Brenes and Y. Huang. Your model is predictive---but is it useful? Theoretical and empirical considerations of a new paradigm for adaptive tutoring evaluation. In *Proceedings of the 8th International Conference on Educational Data Mining*, 2015. https://www.educationaldatamining.org/EDM2015/uploads/papers/paper_113.pdf.

[9] S. Doroudi, V. Aleven, and E. Brunskill. Robust evaluation matrix: Towards a more principled offline exploration of instructional policies. In *Learning @ Scale*, pp. 3--12, 2017. https://doi.org/10.1145/3051457.3051463.

[10] Y. Ozyurt, T. Almaci, S. Feuerriegel, and M. Sachan. Personalized exercise recommendation with semantically-grounded knowledge tracing. In *Advances in Neural Information Processing Systems*, vol. 38, pp. 13019--13059, 2025. https://doi.org/10.52202/085713-0440.

[11] X. Cheng, G. Zhuang, Y. Xie, J. Wang, Y. Liu, Q. Guan, L. Fang, and S. Pan. UniER: A unified benchmark for item-level and path-level exercise recommendation. *arXiv:2605.16750v1*, 2026. https://arxiv.org/abs/2605.16750v1.

[12] Y. Sun and Z. Liu. ACcoding-Dataset: v2.0.0. Zenodo dataset record published 14 July 2026. https://doi.org/10.5281/zenodo.11395043.

[13] Institute of Computing, Federal University of Amazonas. CodeBench dataset, release 1.81, semester archives 2016-1 and 2016-2. Retrieved 26 September 2026. https://codebench.icomp.ufam.edu.br/dataset/.

[14] N. Hoa and V. K. Le. Step-level evidence integration in bayesian knowledge tracing: A controlled within-model ablation study. *Mendeley Data*, version 1, 22 September 2026. Dataset/notebook metadata inspected; notebook and peer-reviewed publication not verified. https://doi.org/10.17632/tnzghkdgfw.1.

[15] F. Zhao. Correctness is not homogeneous evidence: A correctness-conditioned evidence-aware knowledge tracing model. *arXiv:2608.22267v1*, 2026. https://arxiv.org/abs/2608.22267v1.

[16] Y. Zhang, X. Meng, and J. Lin. Augmenting knowledge tracing with self-regulated learning indicators for reliable skill-mastery estimation. In *Learning @ Scale*, pp. 373--384, 2026. Metadata and abstract inspected. https://doi.org/10.1145/3774398.3811597.

[17] S. Pu and L. Becker. Self-attention in knowledge tracing: Why it works. In *Artificial Intelligence in Education*, vol. 13355, pp. 731--736, 2022. https://doi.org/10.1007/978-3-031-11644-5_75.

[18] C. Wang, S. Sahebi, S. Zhao, P. Brusilovsky, and L. O. Moraes. Knowledge tracing for complex problem solving: Granular rank-based tensor factorization. In *User Modeling, Adaptation and Personalization*, pp. 179--188, 2021. https://doi.org/10.1145/3450613.3456831.

[19] M. F. I. Amin, Y. Watanobe, D. M. Muepu, K. Nanaumi, H. Suzuki, M. S. Mia, and M. M. Rahman. Revision-aware success prediction from multi-attempt programming trajectories. *arXiv:2608.26169v1*, 2026. Preprint; source dates are inconsistent, so only the year is cited. https://arxiv.org/abs/2608.26169v1.

[20] M. Pankiewicz, Y. Shi, and R. S. Baker. srcML-DKT: Enhancing deep knowledge tracing with robust code representations from srcML. In *Proceedings of the 18th International Conference on Educational Data Mining*, pp. 541--548, 2025. https://doi.org/10.5281/zenodo.15870306.

[21] Y. Liang, T. Peng, Y. Pu, and W. Wu. HELP-DKT: An interpretable cognitive model of how students learn programming based on deep knowledge tracing. *Scientific Reports*, 12:4012, 2022. https://doi.org/10.1038/s41598-022-07956-0.

[22] N. Liu, Z. Wang, R. G. Baraniuk, and A. S. Lan. Open-ended knowledge tracing for programming exercises. In *6th CSEDM Workshop*, 2022. Workshop version. https://doi.org/10.5281/zenodo.7157118.